Lab Assignment 3: Comment-Aware Lexical Token Classifier

Course: CS3104 - Compiler Design Laboratory
Syllabus Unit: Unit 2 — Lexical Analysis (Scanning) using Flex
Date: 20/08/2026

Instructions

- Write your name and roll number on top of your source code as comments.
- Source file name must be "Lab3.l".
- Your program must compile cleanly with: flex Lab3.l && gcc lex.yy.c -o lab3 -lfl
- Submit only the source file (Lab3.l) through the same submission portal used for previous assignments - http://10.22.11.15/~abyaym/2026/CS3104_26/submission/

Problem Statement

A lexical analyzer does not see comments the way a human reader does — comments are not tokens at all; they must be recognized and discarded before the scanner classifies the remaining program text into tokens. In this assignment you will combine two responsibilities that are usually taught separately — comment removal and token classification — into a single Flex specification, exactly as a production scanner does in one pass.

Your program must read a C-like source snippet from standard input (or a file) and do the following in a single pass: (1) silently discard everything inside single-line (// ...) and multi-line (/* ... */) comments — none of that text should be printed or classified; and (2) for every remaining lexeme that is not a comment, print the lexeme together with its category: KEYWORD, IDENTIFIER, NUMBER, OPERATOR, or PUNCTUATION.

Example

```
Input:
int sum(int a, int b) {   // adds two integers
    int result;
    result = a + b;   /* store the total
                         before returning */
    return result;
}

Output (excerpt):
int          -> KEYWORD
sum          -> IDENTIFIER
(            -> PUNCTUATION
int          -> KEYWORD
a            -> IDENTIFIER
,            -> PUNCTUATION
...
result       -> IDENTIFIER
=            -> OPERATOR
a            -> IDENTIFIER
+            -> OPERATOR
b            -> IDENTIFIER
```

```
;          -> PUNCTUATION
return      -> KEYWORD
result      -> IDENTIFIER
;          -> PUNCTUATION
}          -> PUNCTUATION
```

Notice that the comment text ("adds two integers", "store the total before returning") never appears in the output — it is consumed and discarded before classification.

Core Requirements

- Comment skipping: recognize // .* for single-line comments and /* ... */ for multi-line comments, and discard both entirely (no output, no classification).
- Use Flex's exclusive start-condition mechanism (%x) to enter a COMMENT state on /* and return to the normal state on */, so that any character — including quotes, braces, or another // — is safely ignored while inside a comment.
- Keyword recognition: treat int, float, char, double, if, else, while, for, return, void as KEYWORD.
- Identifier recognition: [a-zA-Z_][a-zA-Z0-9_]* not matching a keyword is an IDENTIFIER.
- Number recognition: integers and simple decimals (e.g. 42, 3.14) are NUMBER.
- Operator recognition: +, -, *, /, =, <, >, plus the two-character operators ==, <=, >=, != as OPERATOR — list multi-character operator patterns before their single-character prefixes so the longest match is chosen correctly.
- Punctuation recognition: ; , ( ) { } as PUNCTUATION.
- For every accepted token, print it in the form lexeme -> CATEGORY on its own line, in the order it appears in the input.

Practice Problems

1. Trace by hand which tokens your program should print for a snippet containing one single-line comment and one multi-line comment placed back-to-back; verify your program's output matches your hand trace, including that no comment text leaks into the output.
2. Feed your program an input with an unterminated multi-line comment (a /* with no matching */) and observe what happens. Add a rule using <<EOF>> inside your COMMENT start condition that prints "Error: unterminated comment" instead of silently reaching end of file.

Implementation Guidelines

- Keep the comment-skipping rules and the token-classification rules in the same rules section, but make sure the /* rule and its <COMMENT> sub-rules are written before the generic token rules, so a comment is never partially matched as code.
- Declare the exclusive state with %x COMMENT in the definitions section, and use BEGIN(COMMENT) / BEGIN(INITIAL) to switch states.
- Inside the COMMENT state, add a rule for \n that does nothing (or just consumes it) so multi-line comments spanning several lines are fully consumed.
- Order your keyword alternatives before the generic identifier pattern — otherwise a keyword like "int" will be classified as an IDENTIFIER, since Flex prefers the first rule listed when two rules match the same length of input.
- Order two-character operator patterns (==, <=, >=, !=) before their single-character counterparts (=, <, >) for the same reason.

Compilation & Testing

```
$ flex Lab3.l
$ gcc lex.yy.c -o lab3 -lfl
$ ./lab3 < input.c
```

Deliverables

- Lab3.l